

AWS S3 - Complete Learning Guide

Starting From Scratch → Production Ready

PART 1: CORE CONCEPTS & DEFINITIONS

1. Bucket Architecture & Global Namespace

What is S3?

S3 stands for **Simple Storage Service**. It is AWS's **object storage service** — meaning it stores files (called objects) in containers (called buckets).

Think of it like this:

Your Computer:		S3 Equivalent:
— Hard Drive	→	AWS S3 Service
— Folder	→	Bucket
— file	→	Object

But S3 is NOT a file system. It has no real folders — just flat storage with naming tricks to simulate folders.

What is a Bucket?

A bucket is the **top-level container** in S3. Everything lives inside a bucket.

Key rules about buckets:

Rule 1: Bucket names are **globally unique**
(No two people **in** the entire world can have same bucket **name**)

Rule 2: Bucket **name** must be 3-63 characters long

Rule 3: **Only** lowercase letters, numbers, **and** hyphens allowed

Rule 4: Cannot **start or end with** a hyphen

Rule 5: Cannot be formatted **like** an IP address (e.g., 192.168.1.1)

Rule 6: Bucket **is** created **in** a specific AWS Region
(but the **name is** still globally **unique**)

Real world example:

Good bucket names:

- ✓ my-company-logs-2024
- ✓ john-project-data
- ✓ webapp-user-uploads

Bad bucket names:

- ✗ My_Bucket (uppercase + underscore)
- ✗ 192.168.1.1 (looks like IP)
- ✗ ab (too short)
- ✗ my bucket (space not allowed)

Global Namespace Explained

When someone says S3 has a **global namespace**, it means:

Person A in India creates: my-data-bucket ✓ (created successfully)

Person B in USA tries to create: my-data-bucket ✗ (ALREADY TAKEN)

Even though they are **in** different countries and different **AWS** accounts,
the bucket name is taken globally.

This is why naming conventions matter in companies:

Common pattern: {company}-{team}-{purpose}-{environment}

Example: netflix-data-logs-production
netflix-ml-models-staging

Bucket Region

Even though the name is global, the **data physically lives in one region**.

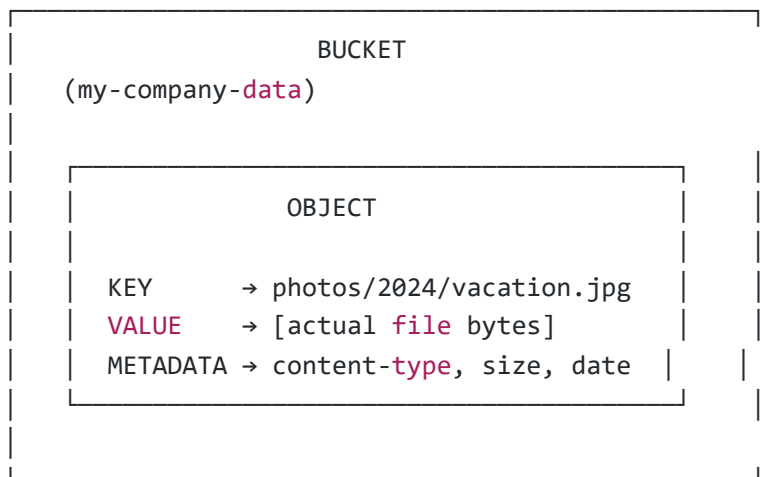
us-east-1 (Virginia) → Your bucket data lives here
eu-west-1 (Ireland) → A different bucket's data lives here

If your users are in Europe and data is in Virginia → Slow access

Solution → Create bucket in eu-west-1 → Faster access

2. Object Storage Model

The Four Core Components



Object - What It Really Is

An object is just **any file** stored in S3.

Can store:

- ✓ Images (.jpg, .png)
- ✓ Videos (.mp4)
- ✓ Documents (.pdf, .docx)
- ✓ Code files (.py, .js)
- ✓ Compressed files (.zip, .tar)
- ✓ Log files (.log, .txt)
- ✓ Database backups (.sql, .dump)
- ✓ JSON/CSV data files

Object size limits:

Minimum: 0 bytes (yes, empty file!)

Maximum: 5 TB per single **object**

Key - The "Path" of an Object

The **key** is the **unique identifier** of an object within a bucket. It's the full "path" to your file.

Bucket: my-company-data

Key examples:

"resume.pdf"	← object at root level
"documents/resume.pdf"	← looks like it's in a folder
"photos/2024/july/vacation.jpg"	← looks like nested folders
"logs/2024/07/15/server.log"	← deeply nested path

IMPORTANT: There are NO actual folders in S3!

The "/" is just part of the key name.

S3 shows them as folders in the console but they don't really exist.

Proof that folders don't exist:

If you delete all objects that start with "photos/"

The "photos" folder disappears automatically

Because it was never a real folder – just a prefix in the key name

Metadata - Data About Your Data

Every object has metadata — information that describes the object.

Two types of metadata:

1. SYSTEM METADATA (set by AWS automatically):

- └─ Content-Type: image/jpeg
- └─ Content-Length: 245678 (file size in bytes)
- └─ Last-Modified: 2024-07-15T10:30:00Z
- └─ ETag: "abc123def456" (unique hash of file content)
- └─ x-amz-storage-class: STANDARD

2. USER-DEFINED METADATA (you set these):

- └─ x-amz-meta-author: "John"
- └─ x-amz-meta-project: "website-v2"
- └─ x-amz-meta-department: "engineering"

(Must start with x-amz-meta-)

Practical example:

You upload a company logo:

Key: assets/logo.png

System Metadata:

Content-Type: image/png

Content-Length: 15234

Last-Modified: 2024-07-15

User Metadata:

x-amz-meta-designer: "Sarah"
x-amz-meta-version: "3.0"
x-amz-meta-approved: "yes"

Object URL Structure

Every object gets a URL you can use to access it:

`https://{bucket-name}.s3.{region}.amazonaws.com/{key}`

Example:

`https://my-company-data.s3.us-east-1.amazonaws.com/photos/2024/vacation.jpg`

Breaking it down:

my-company-data → bucket name

s3.us-east-1.amazonaws.com → S3 endpoint in that region

photos/2024/vacation.jpg → the object KEY

3. Storage Classes

Why Do Storage Classes Exist?

Not all data is accessed equally:

Data you access every day → Needs fast access, you pay more
Data you access once a month → Can be slower, you pay less
Data you access once a year → Can be very slow, you pay very little
Data for legal compliance → May never access, pay almost nothing

Storage Classes let you match access patterns to cost

Storage Class Overview

Storage Class	Availability	Retrieval	Use Case
S3 Standard	99.99%	Milliseconds	Frequently accessed
S3 Standard-IA	99.9%	Milliseconds	Monthly access

S3 One Zone-IA	99.5%	Milliseconds	Recreatable data
S3 Glacier Instant	99.9%	Milliseconds	Quarterly access
S3 Glacier Flex	99.99%	Minutes-Hours	Yearly access
S3 Glacier Deep	99.99%	12-48 Hours	7+ year compliance
S3 Intelligent-Tiering	99.9%	Milliseconds	Unknown access pattern

Each Storage Class - Detailed

S3 Standard

Best **for**: Your main working data **that** you access regularly

- Website images, videos
- Application data files
- Active database backups

Cost structure:

- Higher storage cost per GB
- No retrieval fee
- No minimum storage duration

Think **of it as**: Your desk – things you use **every** day

S3 Standard-IA (Infrequent Access)

Best for: Data you don't access daily but need quickly when you **do**

- Disaster **recovery** files
- **Backup** copies **of** **databases**
- Monthly reports

Cost structure:

- **Lower storage cost than** Standard
- RETRIEVAL FEE per GB retrieved
- **Minimum 30-day storage** charge

Think **of it as**: Filing cabinet – nearby but you don't **open it daily**

WARNING: If you access this data frequently, it becomes **MORE expensive than Standard because of retrieval fees**

S3 One Zone-IA

Best for: Data that can be recreated if lost, infrequently accessed

- Thumbnails (can regenerate from original)
- Processed/transformed data (reprocess if needed)

Cost structure:

- Even cheaper than Standard-IA
- Only stored in ONE availability zone (risky!)
- Retrieval fee applies

Risk: If that one zone has an outage, data could be lost
Standard and Standard-IA store in 3+ availability zones

S3 Glacier Instant Retrieval

Best for: Archive data accessed once per quarter

- Medical records
- News media archives
- Old project files you rarely need

Cost structure:

- Much cheaper storage
- Higher retrieval cost
- Minimum 90-day storage charge
- Still millisecond retrieval (unlike old Glacier)

S3 Glacier Flexible Retrieval

Best for: True archives, once or twice per year access

- Annual compliance data
- Scientific data
- Long-term backups

Retrieval options:

Expedited:	1-5 minutes	(costs more)
Standard:	3-5 hours	(normal cost)
Bulk:	5-12 hours	(cheapest)

Minimum storage: 90 days

S3 Glacier Deep Archive

Best for: Long-term regulatory compliance, 7-10 year retention

- Healthcare records (HIPAA requires 7 years)
- Financial records
- Legal documents

Retrieval:

Standard: 12 hours

Bulk: 48 hours

Minimum storage: 180 days

This **is** the CHEAPEST S3 storage **class**

S3 Intelligent-Tiering

Best for: Data where you don't know the access pattern

- New applications (unsure how users will behave)
- Large datasets **with** unpredictable **access**

How it works automatically:

Access frequently? → Moves **to** Frequent **Access tier**

No access for 30 days? → Moves **to** Infrequent **Access tier**

No access for 90 days? → Moves **to** Archive **Instant Access tier**

No access for 180 days? → Moves **to** Deep **Archive tier** (optional)

Cost: Small monthly fee per **object for monitoring**

Worth it **if access** pattern **is** truly **unknown**

4. Data Encryption

Why Encrypt in S3?

Without encryption:

If someone unauthorized accesses your S3 bucket → They can READ your files

With encryption:

If someone unauthorized accesses your S3 **bucket** → They see SCRAMBLED **DATA**

They need the encryption **KEY to read** it

Three Encryption Types

SSE-S3 (Server-Side Encryption with S3 Managed Keys)

"AWS handles everything for you"

How **it** works:

1. You upload file to S3
2. S3 automatically encrypts **it using** AES-256 algorithm

3. AWS manages **and** stores **the** encryption key
4. When you download, S3 automatically decrypts **it**

You never see **or** touch **the** keys

Analogy: A bank locker where bank keeps **the** master key
(you trust **the** bank to protect **it**)

Best **for**: Basic encryption requirement, no special compliance needs
Header to enable: x-amz-server-side-encryption: AES256

SSE-KMS (Server-Side Encryption with AWS Key Management Service)

"You manage the keys through AWS KMS service"

How it works:

1. You **create** a **key in** AWS KMS (**Key Management Service**)
2. **When** uploading, you specify which KMS **key to use**
3. S3 asks KMS **to encrypt** your **data**
4. KMS **key usage is** logged **in** CloudTrail (**audit** trail!)
5. You can control **WHO** can **use** the **key**

Benefits **over** SSE-S3:

- ✓ **Audit** trail - you can see who decrypted what **and when**
- ✓ **Key** rotation - automatically **change keys** periodically
- ✓ **Access** control - you decide who can **use** the **key**
- ✓ Separate permissions - even **if** someone has S3 **access**, they need KMS permission too

Analogy: Bank locker **where** YOU hold the **master key**,
but bank provides the locker service

Best **for**: Compliance requirements, sensitive **data**, **audit** needs
Header: x-amz-**server**-side-encryption: aws:kms

Client-Side Encryption

"You **encrypt BEFORE** sending to S3"

How **it** works:

1. Your application encrypts **the** file
2. Encrypted file **is** uploaded **to** S3
3. S3 stores already-encrypted data
4. When downloading, your application decrypts **it**
5. AWS never sees **the** unencrypted data

You are fully responsible **for**:

- Managing encryption keys
- Encryption/decryption logic

- Key storage **and** security

Analogy: You **put** your files **in** a locked box, THEN **put that** locked box **in the** bank. Bank never knows what's inside.

Best **for**: When you cannot trust AWS **with** your keys **at** all
Highest security, highest complexity

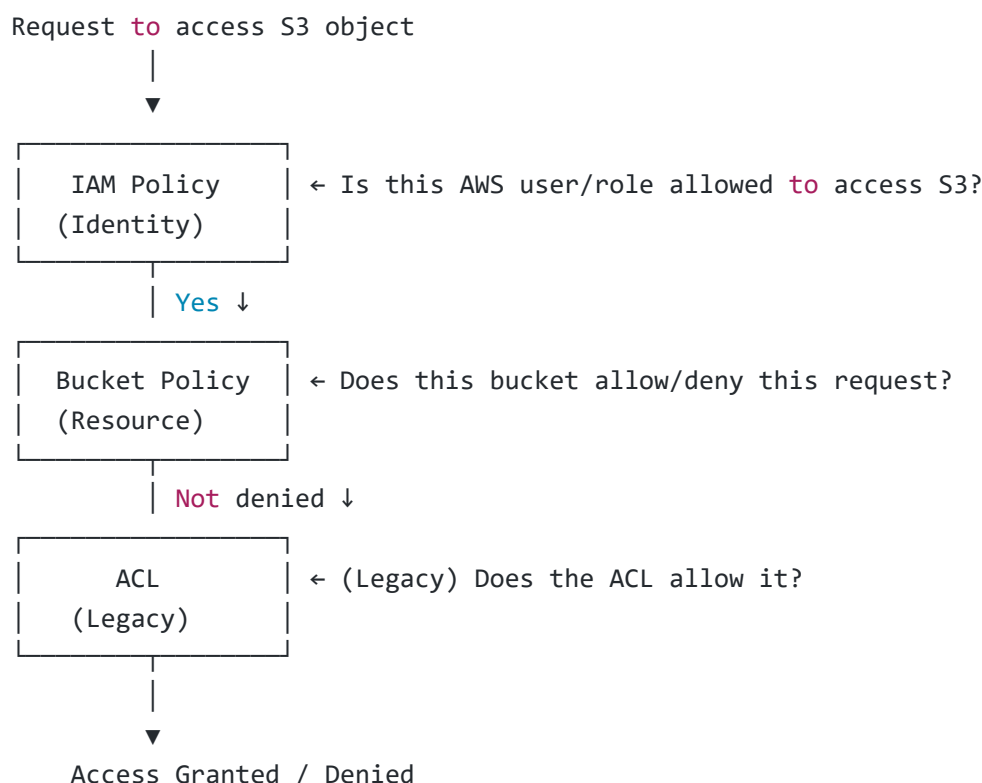
Encryption Comparison

	SSE-S3	SSE-KMS	Client-Side
Who manages key	AWS	You (via KMS)	You
Audit trail	✗ No	✓ Yes	✓ Yes (your system)
AWS sees data	Yes	Yes	✗ Never
Complexity	Simple	Medium	Complex
Extra cost	No	Yes (KMS fee)	No (your infra)
Compliance ready	Basic	High	Highest

5. Access Control Layers

The 3 Ways to Control S3 Access

Think of S3 access like a building with multiple security layers:



IAM Policies (Identity-Based)

Attached **TO**: An IAM User, IAM Group, **or** IAM Role

Controls: What actions that identity can **do**

Example scenario:

Your company has 3 teams:

- Dev team → Can read/write **to** dev-bucket only
- Ops team → Can read all buckets but write **to** none
- Admin team → Full access **to** all buckets

You create IAM policies **for** each group

Example IAM Policy - Read Only Access to specific bucket:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::my-company-data",
        "arn:aws:s3:::my-company-data/*"
      ]
    }
  ]
}
```

Breaking this down:

"Effect": "Allow"	→ This is a permission GRANT
"Action": [...]	→ What operations are allowed
s3:GetObject	→ Download/read objects
s3:ListBucket	→ See what's in the bucket
"Resource": [...]	→ Which resources this applies to
arn:aws:s3:::my-company-data	→ The bucket itself
arn:aws:s3:::my-company-data/*	→ Everything inside the bucket

Bucket Policy (Resource-Based)

Attached **TO**: The Bucket itself

Controls: Who can access this bucket (any AWS account, public, etc.)

Key difference **from** IAM:

IAM Policy → "What can this USER do?"

Bucket Policy → "Who can access this BUCKET?"

Bucket Policies are powerful because:

- ✓ Can grant access **to** OTHER AWS accounts
- ✓ Can grant public access (make bucket public)
- ✓ Can DENY even the bucket owner (explicit deny)
- ✓ Can restrict based on conditions (IP, time, VPC, etc.)

Example Bucket Policy - Make bucket public read:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadAccess",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3::my-public-website/*"
    }
  ]
}
```

New fields:

"Sid" → Statement ID (just a **label**/description)

"Principal" → WHO **is** allowed? "*" means EVERYONE **in** the world

Could also be: "arn:aws:iam::123456789:user/Bob"

Example Bucket Policy - Allow only from specific IP:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3::confidential-bucket",
        "arn:aws:s3::confidential-bucket/*"
      ],
      "Condition": {
        "NotIpAddress": {

```

```

    "aws:SourceIp": "203.0.113.0/24"
  }
}
]
}

```

This says: DENY everyone UNLESS they're coming from IP range 203.0.113.0/24
(your office network for example)

ACL (Access Control List) - Why You Should AVOID It

What ACLs are:

Old way (pre-2006) of controlling S3 access
Grant basic read/write permissions on buckets/objects

Why ACLs are problematic:

- ✗ Very limited – only 5 types of permissions
- ✗ Cannot set conditions (IP, time, etc.)
- ✗ Harder to audit and understand
- ✗ Can create confusing permission conflicts
- ✗ AWS itself recommends DISABLING them

Modern approach:

Use IAM Policies + Bucket Policies ONLY
Disable ACLs on all new buckets (it's now the default!)

When would you see ACLs?

Old legacy systems
Exam questions (to know what NOT to use)
Cross-account object ownership scenarios (rare)

AWS Now Disables ACLs by Default:

When creating a bucket, "Object Ownership" is set to:

"Bucket owner enforced" → ACLs are disabled

This is the correct/recommended setting.

Leave it as default unless you have a very specific reason.

IAM vs Bucket Policy Decision Guide

Question → Which should I use?

I want to control what an AWS user CAN DO across multiple services
→ Use IAM Policy

I want to control who can access MY BUCKET (including other accounts)
→ Use Bucket Policy

I want to make my bucket publicly accessible
→ Use Bucket Policy (Principal: "*")

I want to give EC2 instance access to S3
→ Use IAM Role attached to EC2

I want to DENY access based on conditions (IP, VPC, etc.)
→ Use Bucket Policy (Deny statements are powerful here)

Both can work? → Use BOTH (they stack together)

6. Versioning Fundamentals

What is Versioning?

Without versioning:

You upload "report.pdf"

You upload "report.pdf" again → OVERWRITES the first one

First version is GONE FOREVER

With versioning:

You upload "report.pdf" → Version ID: abc123

You upload "report.pdf" again → Version ID: def456

BOTH versions exist! You can recover the first one.

Versioning States

Three states a bucket can be in:

1. UNVERSIONED (default when you create bucket)
Objects have no version ID (shows as "null")

2. VERSIONING ENABLED
All new objects get unique version IDs
Old objects show version ID as "null"

3. VERSIONING SUSPENDED

You can pause versioning

Existing **versions** are preserved

New objects get **"null"** version ID

(You cannot **go from** suspended back **to** unversioned)

State transitions:

Unversioned → Enabled  (one way!)

Enabled → Suspended 

Suspended → Enabled 

Enabled → Unversioned  (NOT POSSIBLE)

Suspended → Unversioned  (NOT POSSIBLE)

How Versioning Works Practically

Scenario: You have versioning enabled on "my-bucket"

Step 1: Upload **report.pdf**

Bucket: my-bucket

└─ **report.pdf** (**Version:** abc123) ← CURRENT **VERSION**

Step 2: Upload **report.pdf** again (updated **file**)

Bucket: my-bucket

└─ **report.pdf** (**Version:** def456) ← CURRENT **VERSION** (latest)

└─ **report.pdf** (**Version:** abc123) ← OLDER **VERSION**

Step 3: Upload **report.pdf** again

Bucket: my-bucket

└─ **report.pdf** (**Version:** ghi789) ← CURRENT **VERSION**

└─ **report.pdf** (**Version:** def456)

└─ **report.pdf** (**Version:** abc123) ← OLDEST **VERSION**

When you request **"report.pdf"** without specifying **version:**

→ You get ghi789 (the latest)

When you request **"report.pdf"** with **version** ID abc123:

→ You get the original **file**

To recover old **version:**

→ Just delete the newer versions or download the old **one**

Delete Markers

What happens when you **DELETE** a versioned **object**?

S3 does **NOT** actually **delete** the **object**!

Instead, it places a "DELETE MARKER" on top.

Before delete:

```
report.pdf (Version: ghi789) ← CURRENT
report.pdf (Version: def456)
report.pdf (Version: abc123)
```

After you delete report.pdf:

```
report.pdf (DELETE MARKER) ← CURRENT (this makes it "invisible")
report.pdf (Version: ghi789)
report.pdf (Version: def456)
report.pdf (Version: abc123)
```

Now if you request report.pdf → You get 404 (Not Found)
But the data is NOT gone!

To UNDO the delete:

Simply delete the DELETE MARKER
→ The latest version (ghi789) becomes visible again

This is how S3 provides "undelete" functionality!

MFA Delete

What is MFA Delete?

An extra security layer for versioning operations
Requires your MFA device to perform certain actions

What requires MFA when MFA Delete is enabled?

1. Permanently deleting a specific object version
2. Changing versioning state (enable/suspend)

What does NOT require MFA?

- Regular deletes (just creates delete markers)
- Uploading new versions
- Reading objects

Why use MFA Delete?

Protection against:

- Accidental mass deletion
- Compromised AWS credentials
- Malicious insiders deleting all versions

How to enable:

Only the BUCKET OWNER (root account) can enable MFA Delete
Cannot be done via Console – must use AWS CLI

CLI command:

```
aws s3api put-bucket-versioning \
```

```
--bucket my-bucket \  
--versioning-configuration Status=Enabled,MFADelete=Enabled \  
--mfa "arn:aws:iam::123456:mfa/mydevice 123456"
```

7. Cross-Region Replication (CRR)

What is CRR?

CRR automatically copies objects from a bucket in one region to a bucket in a DIFFERENT region.

Source Bucket		Destination Bucket
us-east-1	→	eu-west-1
(Virginia)		(Ireland)

Every new object uploaded to source → Automatically replicated to destination

Prerequisites for CRR

Before you can enable CRR, you MUST:

1. ☒ Enable VERSIONING on BOTH source AND destination bucket
(Replication doesn't work without versioning)
2. ☒ Create an IAM Role that allows S3 to replicate
(S3 needs permission to read from source, write to destination)
3. ☒ Buckets must be in DIFFERENT regions
(Same region replication is called SRR - Same Region Replication)

CRR Use Cases

Use Case 1: Compliance Requirements

"Data must exist in a specific geographic location"

Company stores data in us-east-1

EU regulation requires copy in eu-west-1

→ CRR ensures automatic EU copy

Use Case 2: Low Latency Access for Global Users

Your users are in Asia and US

Store originals in us-east-1

Replicate to ap-southeast-1

→ Asian users access data from nearby region

Use Case 3: Disaster Recovery

If us-east-1 has a regional outage

Your data is safe in eu-west-1

Applications can failover to EU bucket

Use Case 4: Development vs Production Separation

Production bucket in us-east-1

Dev team bucket in us-west-2

Replicate prod data to dev for testing

(but keep them separate for security)

What Gets Replicated vs What Doesn't

GETS REPLICATED:

- ✓ New objects uploaded after replication enabled
- ✓ Object metadata
- ✓ Tags
- ✓ Object ACLs
- ✓ Encrypted objects (with proper KMS setup)

DOES NOT GET REPLICATED:

- ✗ Objects uploaded BEFORE replication was enabled
(Unless you use S3 Batch Replication for this)
- ✗ Objects in Glacier or Glacier Deep Archive
- ✗ Delete markers (by default, configurable)
- ✗ Lifecycle policy deletions
- ✗ Objects from ANOTHER replication (no chain replication by default)

8. Object Lock

What is Object Lock?

Object Lock prevents objects from being DELETED or MODIFIED for a specified period of time (or forever).

Think of it as: "Write Once Read Many" (WORM) storage

Real world analogy:

Signing a legal document and laminating it

No one can change it – it's fixed in time

Two Modes of Object Lock

Governance Mode

GOVERNANCE MODE

"Most users cannot delete, but privileged admins can override"

Regular users:

- ✗ Cannot delete the object
- ✗ Cannot modify the object

Admin users with special IAM permission:

- ✓ CAN override the lock
- ✓ CAN delete if they have s3:BypassGovernanceRetention permission

Best for:

- Internal compliance requirements
- Where you might need admin override capability
- Testing Object Lock policies before going to Compliance mode

Think of it as: "Protected but with an emergency key that admins have"

Compliance Mode

COMPLIANCE MODE

"NOBODY can delete this – not even root account, not even AWS"

Regular users:

- ✗ Cannot delete the object
- ✗ Cannot modify the object

Admin users:

- ✗ Cannot override

Root account:

- ✗ Cannot override

AWS itself:

- ✗ Cannot help you delete it (they'll refuse)

The ONLY way to "remove" the lock is to wait for it to expire.

Best for:

- Legal/regulatory compliance (FINRA, SEC, HIPAA)
- Healthcare records that must be kept 7 years
- Financial records
- Any situation where "prove the data wasn't tampered with"

Think of it as: "Concrete vault – once sealed, nobody opens it until timer ends"

Object Lock Retention Types

Two ways to define HOW LONG an object is locked:

1. RETENTION PERIOD

Lock the object for a specific number of days/years

Example: "This object cannot be deleted for 7 years"

After 7 years → Lock expires → Can be deleted

2. LEGAL HOLD

Lock the object INDEFINITELY (no time limit)

Used when: Lawsuit is pending, investigation ongoing

"Lock this until legal team says otherwise"

A user with s3:PutObjectLegalHold permission can turn legal hold ON or OFF anytime
(Independent of retention period)

Object Lock Requirements

Important constraints:

1. Must enable Object Lock when CREATING the bucket
(Cannot enable on existing bucket!)
2. Enabling Object Lock automatically enables Versioning
3. You can set a DEFAULT retention period at bucket level
(Applies to all new objects if not specified per-object)
4. Can set retention at OBJECT level too (overrides bucket default)

9. Lifecycle Policies

What are Lifecycle Policies?

Lifecycle policies automatically move **or** delete objects based **on** their AGE **or** other conditions.

Why needed?

- You can't manually manage millions **of** files
- Data naturally becomes less important **over** time
- Automate **the** cost optimization journey

The Hot → Cold → Archive Strategy

Data temperature concept:

HOT **DATA**: Accessed very frequently

- Store **in** S3 Standard
- Pay more, get fast **access**

WARM **DATA**: Accessed occasionally

- Store **in** S3 Standard-IA
- Pay less, slight retrieval cost

COLD **DATA**: Rarely accessed

- Store **in** Glacier
- Pay very little, slow retrieval

FROZEN **DATA**: Almost never **access**, compliance **only**

- Store **in** Glacier Deep Archive
- Pay almost nothing, very slow retrieval

Lifecycle Policy Example

Real-world scenario: Company log files

Day 0: Log file created → S3 Standard
(Engineers actively **analyze** it)

Day 30: **Move to** S3 Standard-IA
(Analysis done, might need **to reference** occasionally)

Day 90: **Move to** S3 Glacier Flexible Retrieval
(**Archived**, **accessed** maybe once a **year for** audits)

Day 365: **Move to** S3 Glacier Deep **Archive**
(**7-year** compliance requirement, **never** really **accessed**)

Day 2555: DELETE automatically
(7 years passed, compliance period over)

Lifecycle Rule (simplified):

```
{
  "Rules": [
    {
      "Status": "Enabled",
      "Filter": {"Prefix": "logs/"},
      "Transitions": [
        {
          "Days": 30,
          "StorageClass": "STANDARD_IA"
        },
        {
          "Days": 90,
          "StorageClass": "GLACIER"
        },
        {
          "Days": 365,
          "StorageClass": "DEEP_ARCHIVE"
        }
      ],
      "Expiration": {
        "Days": 2555
      }
    }
  ]
}
```

Lifecycle Policy Conditions You Can Filter On

1. PREFIX: Apply only to objects starting with certain text
"logs/" → Only log files
"2024/" → Only 2024 data
2. OBJECT TAGS: Apply based on tags
tag: department=finance → Only finance files
3. OBJECT SIZE:
Minimum size, maximum size
4. COMBINATION: Prefix + Tags together

Minimum Days Between Transitions

S3 has minimum days before transitioning (important **for** exams!):

Standard → Standard-IA:	Minimum 30 days
Standard → Intelligent-Tiering:	Minimum 0 days
Standard → Glacier Instant:	Minimum 90 days
Standard → Glacier Flexible:	Minimum 90 days
Standard → Deep Archive:	Minimum 90 days

Standard-IA → Glacier Flexible: Minimum 30 days

Note: Cannot transition FROM Glacier BACK to Standard via lifecycle
(Must restore/download **and** re-upload)

10. Intelligent-Tiering Deep Dive

How It Actually Works

Intelligent-Tiering monitors each object's **access** pattern individually.

It has multiple **access** tiers within it:

Tier 1: Frequent **Access**

Default tier **for all new** objects

Same cost as S3 Standard

Tier 2: Infrequent **Access** (**after** 30 days **of** no **access**)

Automatically moved here

~40% cheaper storage cost

Tier 3: Archive Instant **Access** (**after** 90 days **of** no **access**)

Automatically moved here

~68% cheaper than Standard

Tier 4: Deep Archive (**after** 180 days, optional)

Must explicitly activate this tier

~95% cheaper than Standard

12-hour retrieval (like Glacier Deep Archive)

If you **access** an object **in** IA tier → immediately moves back **to** FA tier

Cost Pattern of Intelligent-Tiering

You PAY for:

✓ Storage in whatever tier the object lands in

- ✓ Monitoring fee: \$0.0025 per 1,000 objects per month

You **DO NOT** pay for:

- ✓ No retrieval fees (unlike Standard-IA)
- ✓ No transition fees **between** tiers
- ✓ No **minimum storage duration** penalty

When **is** it **NOT** worth it?

- ✗ Objects smaller **than** 128 KB
(Monitoring fee > storage savings **for** tiny files)
- ✗ Objects you know will **always** be **accessed** frequently
(Just **use** Standard, **no** monitoring fee)
- ✗ Very few objects
(Monitoring cost negligible vs savings **only with** many objects)

Sweet spot: **Large** numbers **of** objects **with** UNPREDICTABLE **access** patterns

11. Replication Rules and Ownership Controls

Replication Rules Deep Dive

You can have MULTIPLE replication rules per bucket:

Rule 1: Replicate **"logs/"** prefix → us-west-2

Rule 2: Replicate **"images/"** prefix → eu-west-1

Rule 3: Replicate objects tagged **"critical"** → ap-southeast-1

Priority matters when rules conflict:

Rule with HIGHER priority number wins

Priority 1 is HIGHEST (counterintuitive!)

Actually: Priority 0 = lowest, higher number = higher priority

Ownership Controls Issue

Problem **scenario**:

Company A owns source **bucket** in us-east-1

Company B owns destination **bucket** in eu-west-1

Company A replicates objects to Company B's **bucket**

WHO OWNS the replicated objects?

Default: The **ORIGINAL** uploader owns them (Company A)

Problem: Company B owns the bucket but NOT the objects inside!
Company B cannot delete or manage those objects!

Solution: Replica Ownership Override

Configure replication to automatically transfer ownership to the DESTINATION BUCKET OWNER.

Setting: "Replica Ownership Override" → "Destination bucket owner"

Now: Company A uploads → Replicates → Company B owns the copy
Company B can manage, delete, set lifecycle on their copy

Replication Time Control (RTC)

By default: Replication happens "eventually" (usually within 15 min)
No SLA guarantee

With RTC (Replication Time Control):

AWS guarantees 99.99% of objects replicate within 15 MINUTES

You get CloudWatch metrics for replication

Costs extra (\$0.015 per GB)

Use when: You need guaranteed replication timing (compliance, sync)

12. S3 Best Practices

Cost Optimization

1. Use Lifecycle Policies aggressively
Don't manually manage storage classes
2. Use Intelligent-Tiering for unpredictable data
Save cost without managing tiers manually
3. Enable S3 Storage Lens
Dashboard showing where you're spending money
4. Use S3 Inventory
CSV report of all objects and their storage classes
Find objects stuck in wrong (expensive) class
5. Enable Requester Pays

Bucket owner pays storage
REQUESTER (downloader) pays data transfer costs
Useful for large public datasets

6. Multipart Upload for large files
For files > 100 MB: use multipart upload
For files > 5 GB: multipart is REQUIRED
Saves money if upload fails (resume vs restart)

Compliance Best Practices

1. Always encrypt sensitive data
Use SSE-KMS for audit trail
2. Enable Object Lock for immutable records
Compliance mode for regulatory requirements
3. Enable CloudTrail S3 data events
Logs every GetObject, PutObject, DeleteObject
4. Enable Server Access Logging
Records all requests to a bucket
5. Use Bucket Policies to deny HTTP (only allow HTTPS)
Condition: aws:SecureTransport = false → Deny
6. Disable ACLs on all buckets
Use IAM + Bucket Policies instead
7. Enable MFA Delete for critical buckets
8. Never use root account for S3 operations

Security Best Practices

1. Block Public Access settings
Enable ALL 4 block public access settings by default
Only disable if you intentionally need public bucket
2. Principle of Least Privilege
Give minimum permissions needed
No wildcard actions (s3:*) unless absolutely necessary
3. VPC Endpoint for internal access
If EC2 accesses S3, use VPC Endpoint
Traffic stays within AWS network (doesn't go to internet)

4. Bucket Policy deny HTTP

```
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:*",
  "Condition": {
    "Bool": {"aws:SecureTransport": "false"}
  }
}
```
5. Regularly audit bucket policies and IAM policies
Use IAM Access Analyzer for S3
Flags buckets accessible outside your account

PART 2: PRACTICE TASKS

Task 1: Bucket Creation with Proper Naming Convention

Objective: Create buckets following naming best practices

Steps:

1. Create a bucket named: {yourname}-learning-s3-{random4digits}
Example: john-learning-s3-4821
2. Choose region: us-east-1
3. Keep "Block all public access" ENABLED
4. Keep ACLs DISABLED (Bucket owner enforced)
5. Note the bucket ARN

Verification: Can you access it in S3 console?

Try creating same bucket name again → See the error about global namespace

Task 2: Object Upload and Understanding Keys

Objective: Understand the object model and key structure

Steps:

1. Create 3 text files on your computer:
 - report.txt (write "This is my report")
 - photo_description.txt (write "Photo metadata here")
 - notes.txt (write "My notes")

2. Upload them with these key structures:
 - Upload report.txt → key stays "report.txt" (root level)
 - Upload photo_description.txt → rename to "images/2024/photo_description.txt"
 - Upload notes.txt → rename to "documents/personal/notes.txt"
3. In the console, observe how S3 shows "folders"
4. Click on any object → View its properties:
 - What is the Object URL?
 - What is the storage class?
 - What metadata does it have?
5. Try accessing the Object URL directly in browser
 - It should say "Access Denied" (bucket is private)

Task 3: Storage Classes Hands-On

Objective: Upload objects with different storage classes and compare

Steps:

1. Upload 4 copies of the same file with different storage classes:

Upload file1.txt → S3 Standard (default)
Upload file2.txt → S3 Standard-IA
Upload file3.txt → S3 Glacier Instant Retrieval
Upload file4.txt → S3 Intelligent-Tiering

2. View all 4 objects in console
 - Compare storage class column
3. Calculate theoretical cost (use AWS S3 pricing page):
 - If you stored 1 TB of each for 1 month, what would each cost?
4. Try to download file3.txt (Glacier Instant)
 - Should work immediately!
 - Note: Glacier Instant still has millisecond retrieval

Note: Glacier FLEXIBLE requires restore before download
(Don't use Flexible for this task – it's expensive to restore)

Task 4: Encryption Configuration

Objective: Configure and test all three encryption types

Steps:

1. Create 3 new buckets (or use prefixes in existing bucket):

Bucket 1: Enable SSE-S3 as default encryption

- Go to bucket → Properties → Default Encryption
- Select: SSE-S3 (AES-256)

Bucket 2: Enable SSE-KMS as default encryption

- First go to KMS console → Create a symmetric key
- Name it: "my-s3-encryption-key"
- Go to bucket → Properties → Default Encryption
- Select: SSE-KMS → Choose your key

Bucket 3: No default encryption (client-side simulation)

2. Upload a file to each bucket

3. For KMS bucket:

- Click on uploaded object
- Check Properties → Server-side encryption
- You'll see the KMS key ARN

4. Go to CloudTrail → Event History

- Filter by "kmsKeyId"
- You'll see the decrypt/encrypt calls logged!
- This is the audit trail SSE-KMS provides

5. For client-side simulation:

- Use openssl to encrypt a file locally:
openssl enc -aes-256-cbc -in notes.txt -out notes.enc -k mypassword
- Upload notes.enc to bucket 3
- This simulates client-side encryption (you encrypted before upload)

Task 5: IAM Policy and Bucket Policy

Objective: Set up and test access control

Steps:

PART A - Bucket Policy:

1. Create a bucket: {yourname}-public-test
2. Upload a text file: hello.txt (content: "Hello World")
3. Try to access the Object URL → Should fail (403)

4. Add this bucket policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
```

```

    "Sid": "PublicRead",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::{YOUR-BUCKET-NAME}/*"
  }
]
}

```

NOTE: First disable "Block public access" → then apply policy
(S3 won't let you apply public policy while block is on)

5. Try Object URL again → Should work now! You see "Hello World"

PART B - IP Restriction:

6. Add a second statement to DENY access from non-whitelisted IPs
(Use Condition: NotIpAddress: aws:SourceIp: your actual IP)

Find your IP: <https://whatismyip.com>

PART C - IAM Policy:

7. Create an IAM user (no permissions)
8. Try accessing your private bucket → Should fail
9. Create and attach IAM policy with s3:GetObject and s3:ListBucket
10. Test again → Should work now

Task 6: Versioning and Delete Markers

Objective: Experience versioning behavior hands-on

Steps:

1. Create bucket: {yourname}-versioning-test
2. ENABLE versioning on the bucket
3. Create a file: document.txt with content "Version 1"
4. Upload to bucket
5. Edit file locally: change to "Version 2"
6. Upload SAME filename to bucket
7. Edit file locally: change to "Version 3"
8. Upload SAME filename to bucket
9. In console: Click "Show versions" toggle
→ You should now see 3 versions with different version IDs!
10. Click on oldest version → Download it
→ Open it → Should say "Version 1"

11. Now DELETE the object (without selecting specific version)
 - Object disappears from normal view
12. Show versions again
 - You see a DELETE MARKER at the top!
 - All 3 versions still exist below it
13. Delete the DELETE MARKER
 - Object comes back! (Version 3 is current again)
14. Now delete a SPECIFIC version permanently
 - Select Version 1 → Delete it with version ID
 - Now you have Version 2 and Version 3 only

Reflection questions:

- What's the difference between soft delete and hard delete?
- Why is versioning useful for accidental deletion scenarios?

Task 7: Lifecycle Policy - Hot to Cold

Objective: Create lifecycle policies that automatically manage storage

Steps:

1. Create bucket: {yourname}-lifecycle-test
2. Upload several files with prefix "logs/":
 - logs/app-2024-01.log
 - logs/app-2024-02.log
 - logs/app-2024-03.log

3. Create a lifecycle rule:
Rule name: "log-archival-policy"

Filter: Prefix = "logs/"

Transitions:

- After 30 days → Move to Standard-IA
- After 90 days → Move to Glacier Flexible Retrieval
- After 365 days → Move to Glacier Deep Archive

Expiration:

- After 2555 days → Delete permanently

4. Create ANOTHER rule for "temp/" prefix:

Rule name: "temp-cleanup"

Filter: Prefix = "temp/"

Expiration: After 7 days → Delete

5. Upload something to "temp/" folder
 - In 7 days it will auto-delete (you can observe in console)

6. For versioning-enabled buckets, add:
- Delete expired delete markers: Yes
 - Delete non-current versions after: 30 days
- (Prevents version buildup)

Note: Transitions won't happen immediately in practice
But you can verify rules are configured correctly

Task 8: Object Lock

Objective: Test Governance and Compliance mode

IMPORTANT:

- Object Lock must be enabled when creating bucket
- You CANNOT enable it later on existing buckets
- Compliance mode locks are truly permanent until retention expires
- For testing, use very short retention periods (1-2 days minimum)

Steps:

PART A - Governance Mode:

1. Create NEW bucket: {yourname}-governance-test
 - During creation, check "Enable Object Lock"
 - This also enables versioning automatically
2. Upload a file: protected.txt
3. Set Object Lock on this object:
 - Click object → Object Lock tab
 - Retention mode: GOVERNANCE
 - Retention period: 1 day from now
4. Try to delete protected.txt
 - You'll get an error!
 - "Object is protected by Object Lock"
5. Now try with bypass:
 - If your IAM user has "s3:BypassGovernanceRetention" permission
 - You CAN delete it despite governance mode

PART B - Legal Hold:

6. Upload another file: legal.txt
7. Set Legal Hold: ON
8. Try to delete → Error!
9. Turn Legal Hold: OFF
10. Try to delete → Success! (Legal hold is just on/off, no timer)

PART C - Observe Compliance Mode (READ ONLY - don't set):

- Understand that Compliance **mode** cannot be overridden
- Even AWS cannot **help** you **delete** it
- **Only set in production** when you're **sure about retention periods**

Task 9: Cross-Region Replication

Objective: **Set** up CRR **between** two buckets **in** different regions

Steps:

1. **Create SOURCE bucket in us-east-1:**

Name: {yourname}-crr-source-east

Enable versioning: YES (**REQUIRED** for CRR)

2. **Create DESTINATION bucket in us-west-2:**

Name: {yourname}-crr-destination-west

Enable versioning: YES (**REQUIRED**)

3. **On the SOURCE bucket:**

→ **Go to Management** tab → **Replication Rules** → **Create rule**

Rule name: "replicate-all"

Status: Enabled

Source: All objects in bucket

Destination: Browse and select {yourname}-crr-destination-west

IAM Role: Create new role (let AWS create it)

4. Upload a **NEW** file to source bucket

→ **Wait** 1-5 minutes

→ **Check** destination bucket → **File** should appear!

5. **Check** the **REPLICATION STATUS** of the object in source:

→ Click **object** → **Properties**

→ Should **show "COMPLETED"** under **Replication**

6. Upload 5 files rapidly → **Check** they **all** replicate

7. Optional: Try replicating **to** a **THIRD** bucket in eu-west-1

→ **Create 2 replication rules:** one **to** us-west-2, one **to** eu-west-1

Observations:

- Objects uploaded **BEFORE** enabling CRR **do NOT** replicate
- **Delete** markers **do NOT** replicate by default
- **Check** the IAM **role** created – see what permissions it has

Task 10: S3 Intelligent-Tiering Analysis

Objective: Set up Intelligent-Tiering and understand cost patterns

Steps:

1. Create bucket: {yourname}-intelligent-tier-test
2. Upload 20+ objects with Intelligent-Tiering storage class
(Mix of different sizes if possible)
3. In bucket properties, set default encryption and storage class:
→ Properties → Default storage class
→ Select Intelligent-Tiering
4. Explore the Intelligent-Tiering configuration:
→ You can configure archive tiers
→ Management → Intelligent-Tiering Archives
5. Create an Intelligent-Tiering Archive Configuration:
→ Enable Deep Archive tier after 180 days
6. Use S3 Storage Lens (if available in your account):
→ Go to S3 → Storage Lens → Create dashboard
→ Analyze storage class distribution
7. Calculate:
If you have 100,000 objects at average 1 MB each:
 - Monitoring fee: $100,000 / 1000 * \$0.0025 = \$0.25/\text{month}$
 - Storage in IA tier vs Standard: calculate savings
 - At what point does IT become worth it vs Standard?

Reflection:

- When would you choose IT over Standard?
- What types of workloads benefit most?
- What are the hidden costs?

Task 11: Comprehensive Security Audit

Objective: Apply all security best practices to a bucket

Steps:

1. Create bucket: {yourname}-secure-production
2. Apply these security settings:

ENCRYPTION:

- Enable SSE-KMS with a custom KMS key
- Create the key in KMS console first

BUCKET POLICY (add these statements):

Statement 1 - Deny HTTP:

```
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:*",
  "Resource": [
    "arn:aws:s3:::{bucket}",
    "arn:aws:s3:::{bucket}/*"
  ],
  "Condition": {
    "Bool": {"aws:SecureTransport": "false"}
  }
}
```

Statement 2 - Deny non-encrypted uploads:

```
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:PutObject",
  "Resource": "arn:aws:s3:::{bucket}/*",
  "Condition": {
    "StringNotEquals": {
      "s3:x-amz-server-side-encryption": "aws:kms"
    }
  }
}
```

VERSIONING: Enable

ACCESS:

- Block all public access: ALL 4 settings ON
- Verify ACLs are disabled

LOGGING:

- Enable Server Access Logging
- Create separate bucket for logs: {yourname}-access-logs
- Point logging to this bucket

CLOUDTRAIL (verify):

- Check that S3 data events are being tracked

3. Test:

- Try to upload WITHOUT encryption header → Should fail
- Try to access via http:// → Should fail
- Upload WITH proper KMS encryption → Should succeed

Task 12: Lifecycle + Versioning Combined

Objective: Manage version accumulation **with** lifecycle policies

Problem scenario:

You have a versioned **bucket**

Every **day** you upload 10 **updated** files

After 1 **year** = 3,650 **versions** accumulating

Cost is getting **out of** hand!

Solution: Lifecycle policies **for** version management

Steps:

1. **Create bucket**: {yourname}-**version-lifecycle**
2. **Enable** versioning
3. Simulate **version** buildup:
Upload **"data.csv"** 5 times **with updated content**
4. **Create** lifecycle **rules**:

Rule 1: **"expire-old-versions"**

Apply to: All objects

Transitions for Non-current versions:

After 30 **days** → Standard-IA

After 60 **days** → Glacier

Expiration:

Delete non-current versions **after**: 90 **days**

Rule 2: **"cleanup-delete-markers"**

Apply to: All objects

Expiration:

Delete expired **object delete** markers: Yes

(**When all versions of** an **object are** deleted,
the **delete** marker itself should be cleaned up)

5. Review what will happen:
Day 0: **New version** uploaded → Standard
Day 30: **Old version** → Standard-IA
Day 60: **Older version** → Glacier
Day 90: **Oldest version** → DELETED automatically
6. **Verify rules in** console:
Management → Lifecycle **Rules** → **View** configured **rules**

Task 13: Replication with Ownership and Filtering

Objective: Advanced replication configuration

Steps:

1. Create source bucket in us-east-1 (versioning enabled)
2. Create destination bucket in eu-west-1 (versioning enabled)
3. Add tags to some files:
 Upload file1.txt with tag: environment=production
 Upload file2.txt with tag: environment=development
 Upload file3.txt (no tags)
4. Create replication rule with TAG FILTER:
 Rule: Only replicate objects where tag environment=production

 This means:
 file1.txt → REPLICATES to EU
 file2.txt → DOES NOT replicate
 file3.txt → DOES NOT replicate
5. Configure Replica Ownership Override:
 → In replication rule → Additional Replication Options
 → Enable: "Replica ownership - Change object ownership"
 → Destination account: Your own account (for this exercise)
6. Test: Upload files with and without the production tag
 Verify only production-tagged files appear in destination
7. Check replication status on individual objects
 → Properties → Replication Status
 → Should show COMPLETED for production files
 → Will show nothing for dev files (not in scope of rule)

Task 14: Cost Analysis and Optimization Project

Objective: Analyze a simulated bucket and optimize costs

Scenario:

You have a bucket with 1 million objects (simulate with categories):

1. Create "test" objects in different scenarios:

 uploads/recent/ → Files uploaded last week (Standard is correct)
 uploads/old/ → Files from 2 years ago (should be archived!)
 logs/current/ → Logs from this month (Standard-IA would save money)
 logs/archive/ → Logs from 2 years ago (should be Glacier!)
 assets/images/ → Website images accessed daily (Standard correct)
2. Calculate current monthly cost (hypothetically):
 Assume: 100 GB in each category at Standard pricing
3. Apply appropriate storage classes:
 uploads/old/ → Move to Standard-IA or Glacier

logs/current/ → Move to Standard-IA
logs/archive/ → Move to Glacier Deep Archive
Keep uploads/recent/ and assets/images/ in Standard

4. Calculate NEW cost after optimization:

Use AWS Pricing Calculator:

<https://calculator.aws/pricing/2/home#/>

5. Create lifecycle policies to automate this going forward:

Rule for uploads/: Move to IA after 30 days, Glacier after 90 days

Rule for logs/: Move to IA after 7 days, Glacier after 30 days

6. Document:

Before: \$X/month

After: \$Y/month

Savings: Z%

Task 15: End-to-End Compliance Setup

Objective: Set up a fully compliant S3 configuration

Scenario:

You're setting up S3 for a healthcare company that must:

- Retain medical records for 7 years
- Encrypt all data
- Never allow modification of records
- Have full audit trail
- Optimize costs for old records

Complete setup:

1. Create main bucket: {yourname}-hipaa-records

During creation:

- ✓ Enable Object Lock
- ✓ This auto-enables versioning

2. Apply encryption:

- ✓ SSE-KMS with dedicated KMS key
- ✓ Create KMS key: "hipaa-records-key"

3. Apply bucket policy:

- ✓ Deny HTTP (only HTTPS)
- ✓ Deny non-KMS uploads
- ✓ Deny all public access

4. Configure Object Lock defaults:

- ✓ Default retention: COMPLIANCE mode
- ✓ Retention period: 2557 days (7 years)

5. Set up lifecycle (**AFTER lock period**):
 - Days** 0-365: Standard (actively **accessed**)
 - Days** 365-730: Standard-IA (occasional **access**)
 - Days** 730-2557: Glacier Flexible (compliance **storage**)
 - Day** 2558: **DELETE** (compliance **period over**)
6. **Enable server access logging**
7. **Create CRR to backup** region:
 - ✓ **Source**: us-east-1
 - ✓ **Destination**: us-west-2 (disaster **recovery**)
8. **Test** upload:
 - Upload patient_record.pdf
 - Verify**:
 - Encrypted **with** KMS: ✓
 - Has 7-**year retention lock**: ✓
 - Replicated **to backup** region: ✓
 - Try **to delete** → FAILS (compliance **mode**): ✓
9. **Document** your architecture:
 - Draw a diagram showing:
Upload → Encryption → **Object Lock** → Lifecycle → **Replication**

PART 3: OUT-OF-SYLLABUS BUT IMPORTANT TOPICS

1. S3 Transfer Acceleration

What: Speeds up uploads to S3 by routing through AWS CloudFront edge locations

Normal upload:

Your Computer → Internet → S3 bucket in us-east-1

With Transfer Acceleration:

Your Computer → Nearest CloudFront Edge → AWS backbone → S3

Why faster:

AWS internal network **is** much faster **than public** internet

Edge locations **are** geographically **close to** you

When to use:

Uploading **large** files **from** distant location

Mobile app **users globally** uploading **to** centralized **bucket**

Transfer speeds **are critical**

Cost: Extra per GB transferred

Test tool: <http://s3-accelerate-speedtest.s3-accelerate.amazonaws.com>

2. S3 Event Notifications

What: S3 can trigger notifications when things happen in a bucket

Events you can trigger on:

- s3:ObjectCreated:* (any upload)
- s3:ObjectRemoved:* (any delete)
- s3:ObjectRestore:* (Glacier restore)

Where notifications can go:

- SNS Topic (send email/SMS)
- SQS Queue (trigger processing queue)
- Lambda Function (run code)

Practical example:

User uploads image → S3 Event → Lambda → Resize image →
Save thumbnail back **to** S3

This is a very common serverless pattern!

Why important: Powers most S3-based serverless architectures

3. Pre-Signed URLs

What: Temporary URLs that allow ANYONE to access a private S3 object

Problem it solves:

- Your bucket **is** private
- You want **to** share ONE **file with** a user **for** 1 hour
- You don't want **to** make bucket public

Solution: **Generate** pre-signed URL

- URL contains: bucket, key, expiry time, **and** your credentials (encoded)
- Anyone **with** URL can **access** the **file until** expiry

Example **use case**:

- User clicks "**Download Invoice**" on your website
- Backend generates pre-signed URL (expires **in** 15 min)
- User's browser downloads using that URL
- URL expires → No one **else** can **use** it

CLI example:

```
aws s3 presign s3://my-bucket/invoice.pdf --expires-in 3600
```

Output: Long URL that works **for** 1 hour

Why important: Very common interview question, used **in** almost every app

4. S3 Select and Glacier Select

What: Query data INSIDE S3 objects without downloading the whole file

Problem:

You have a 10 GB CSV file in S3

You need just 3 rows out of 10 million rows

Without S3 **Select**: Download 10 GB → Filter locally

With S3 **Select**: Run **SQL query** on S3 → Download just those 3 **rows**

Example **SQL query**:

```
SELECT * FROM s3object WHERE age > 25 AND city = 'Mumbai'
```

Supported formats: CSV, **JSON**, Parquet

Cost savings:

Less data transferred = **less** money

Less time = faster results

Glacier **Select**: Same concept but **for archived data**

Why important: **Big data** and analytics workflows

5. S3 Batch Operations

What: Run operations on millions of objects at once

Without Batch Operations:

List all objects → Loop through each → Apply operation

For 1 million objects → Takes forever, complex code

With Batch **Operations**:

Give S3 a **list** (from S3 Inventory)

Tell it what **to do** (copy, tag, **restore from** Glacier, etc.)

S3 does it automatically **with** progress **tracking**

Use cases:

- Copy 100 million objects **to new bucket**

- Add tags to all existing objects
- Change storage class of all objects
- Restore 1 million Glacier objects
- Run Lambda on each object (custom processing)

Why important: Real-world companies have HUGE object counts

6. S3 Access Points

What: Named access points that simplify permissions for shared buckets

Problem:

You have ONE bucket with data for 5 different teams
Each team should see only THEIR data
Managing one complex bucket policy gets messy

Solution: Create separate Access Points

Main Bucket: company-data

Access Point 1: finance-access-point

- Finance team uses this to access
- Policy: Only allow access to "finance/*" objects

Access Point 2: engineering-access-point

- Engineering team uses this
- Policy: Only allow access to "engineering/*" objects

Each team has their own "view" into the same bucket
Policies are simpler and separated by team

Why important: Multi-team S3 access is very common in enterprises

7. S3 Multipart Upload

What: Upload large files in multiple parts simultaneously

How it works:

1. Initiate multipart upload → Get upload ID
2. Split file into parts (minimum 5 MB each, max 10,000 parts)
3. Upload each part (can be parallel)
4. Complete upload → S3 assembles all parts

Benefits:

- Can pause and resume uploads

- If one part fails, retry just that part
- Faster (upload multiple parts simultaneously)
- Required **for** files > 5 GB

Requirement:

- > 100 MB → Recommended to use multipart
- > 5 GB → REQUIRED (single PUT limit **is** 5 GB)

AWS CLI does **this** automatically:

aws s3 cp largefile.zip s3://bucket/ → Uses multipart automatically

Why important: Understanding **this is** critical **for** architecture/interviews

8. S3 Inventory

What: Daily or weekly report of all objects in your bucket

Output: CSV or ORC file saved to another S3 bucket

Report contains:

- Object key, size, storage class
- Encryption status
- Replication status
- Object **Lock status**
- **Last** modified date

Why useful:

- Find objects **not** yet encrypted
- Find objects **in** wrong **storage class**
- **Audit Object Lock** compliance
- Feed **into** Batch **Operations**

How often: Daily **or** Weekly

Delivery: **Within** 24-48 **hours** (**not** real-time)

Why important: **Security** audits, compliance, **cost** optimization audits

9. S3 Storage Lens

What: Organization-wide analytics dashboard for S3 usage

Shows you:

- Total storage used across ALL buckets
- Storage by region, bucket, prefix
- Cost breakdown

- Unusual activity (sudden spike **in** deletions?)
- Recommendations **for** savings

Two tiers:

Free: Basic metrics

Paid: Advanced metrics with prefix-level granularity

Why important:

In large organizations with hundreds of buckets,
you need a central view of S3 usage **and** spend

This is different **from** individual bucket metrics!

10. Website Hosting on S3

What: Host a static website directly on S3 (no servers!)

What you can host:

- HTML files
- CSS, JavaScript
- Images
- Any static files

What you cannot host:

- Server-side code (PHP, Python, Node backend)
- Dynamic databases

Setup:

1. **Create bucket** named exactly **like** your **domain**: www.example.com
2. **Enable static** website hosting
3. **Set index.html as index document**
4. **Set error.html as error document**
5. Make **bucket public**
6. Upload your HTML files

URL format:

http://{bucket-name}.s3-website-{region}.amazonaws.com

Use with CloudFront for:

- HTTPS support
- Custom **domain**
- **Global CDN caching**

Why important: Very common architecture, often **in** certification exams

QUICK REFERENCE: Everything On One Page

BUCKET: Container, globally unique name, regional data
OBJECT: File stored in bucket (max 5 TB)
KEY: Full path/name of object
METADATA: Data about the object

STORAGE CLASSES:

Standard → Frequent access, milliseconds retrieval
Standard-IA → Monthly access, retrieval fee, 30 day min
One Zone-IA → Recreatable data, 1 AZ only, cheap
Glacier IR → Quarterly access, milliseconds retrieval
Glacier Flex → Yearly access, minutes-hours retrieval
Deep Archive → 7yr compliance, 12-48 hours retrieval
Intelligent-T → Unknown pattern, auto-tiering

ENCRYPTION:

SSE-S3 → AWS manages keys, simplest
SSE-KMS → You manage keys via KMS, audit trail
Client-Side → You encrypt before upload, AWS never sees plaintext

ACCESS CONTROL:

IAM Policy → Who can **do** what (**identity**-based)
Bucket Policy → Who can **access bucket** (**resource**-based)
ACL → AVOID! Legacy, limited, complex

VERSIONING:

Disabled → **Default**, overwrites happen
Enabled → **All versions** kept, **delete** = **delete** marker
MFA **Delete** → Extra **protection**, requires MFA **for permanent** deletion

REPLICATION (CRR):

Requires versioning **on BOTH** buckets
Different regions
IAM **role required**
New objects only (**not** retroactive)

OBJECT LOCK:

Governance → Admins can override **with** permission
Compliance → NOBODY can override **until** expiry
Legal Hold → **On/off switch**, indefinite **protection**

LIFECYCLE:

Automates moving objects **between storage** classes
Based **on** age, prefix, tags
Hot→Cold→**Archive**→**Delete** pattern

OBJECT LOCK MODES:

Governance: **Admin** override possible
Compliance: Zero override, even AWS can't **help**

Final Tips

For Exams / Interviews:

1. Know when to use **Bucket Policy** vs **IAM Policy**
2. Know the **minimum days for** lifecycle transitions (**30 days**, **90 days**)
3. Know what CRR requires (versioning **both** sides, different regions)
4. Know **delete** markers behavior
5. Know Governance vs Compliance **mode difference**
6. Know SSE-S3 vs SSE-KMS **difference** (**audit trail!**)

For Real Work:

1. **Always enable** versioning on important buckets
2. **Always encrypt** sensitive **data with** SSE-KMS
3. **Always use** lifecycle policies **to save** money
4. **Never use** ACLs on new buckets
5. **Always block public access** unless explicitly needed
6. **Use** Compliance **mode Object Lock for** regulatory requirements